
Table of Contents

.....	1
Problem 1	1
Problem 2	3
Problem 3	6

```
clc
clear all
close all
```

Problem 1

```
%plots the graph of x^2*abs(sin x)-4 for x in [0,4]; uses the Secant method
%to find the smallest positive zero accurate to within 10^-6 using p0 = 3.6
%and p1 = 3.7 (also p0 = 2.8 and p1 = 2.9)
```

```
%function for which we want to approximate root
f = @(x) x.^2.*abs(sin(x))-4;
%plots function on the interval [0,4]
figure(1)
plot(0:0.01:4, f(0:0.01:4), 'b-', 'LineWidth', 2)
xlabel('x'); ylabel('y')

p0 = 3.6; %initial guesses (two required for secant method)
p1 = 3.7;
tolexp = -6; %tolerance exponent
tolerance = 10^tolexp; %stopping criteria tolerance
N = 50; %max number of iterations
```

```
%uses secant method to produce approximation for root
Secant(p0, p1, f, N, tolexp)
```

```
p0 = 2.8;
p1 = 2.9;
```

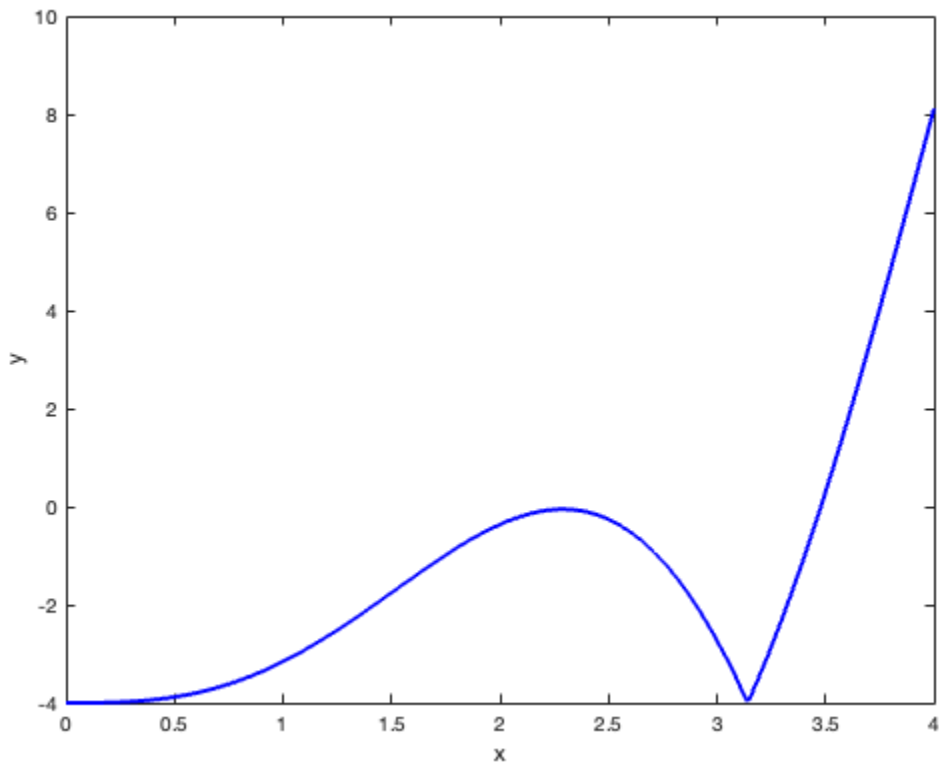
```
Secant(p0, p1, f, N, tolexp) %converges to another zero
```

```
n: 1    p1: 3.7000000    |f(p1)|: 3.25345677    |(p1-p0)/p0|: 0.0277778
n: 2    p2: 3.4857301    |f(p2)|: 0.09933154    |(p2-p1)/p1|: 0.0579108
n: 3    p3: 3.4789822    |f(p3)|: 0.00649957    |(p3-p2)/p2|: 0.0019359
n: 4    p4: 3.4785097    |f(p4)|: 0.00001665    |(p4-p3)/p3|: 0.0001358
n: 5    p5: 3.4785085    |f(p5)|: 0.00000000    |(p5-p4)/p4|: 0.0000003
Tolerance: 10^-6, Approximation: 3.4785085158, Iterations: 5
```

```
n: 1    p1: 2.9000000    |f(p1)|: 1.98791314    |(p1-p0)/p0|: 0.0357143
n: 2    p2: 2.5763517    |f(p2)|: 0.44477977    |(p2-p1)/p1|: 0.1116028
n: 3    p3: 2.4830661    |f(p3)|: 0.22693708    |(p3-p2)/p2|: 0.0362084
n: 4    p4: 2.3858860    |f(p4)|: 0.09609857    |(p4-p3)/p3|: 0.0391371
n: 5    p5: 2.3145089    |f(p5)|: 0.05749731    |(p5-p4)/p4|: 0.0299164
```

n: 6	p6: 2.2081914	f(p6) : 0.08132118	(p6-p5)/p5 : 0.0459352
n: 7	p7: 2.5710991	f(p7) : 0.42998860	(p7-p6)/p6 : 0.1643461
n: 8	p8: 2.1235490	f(p8) : 0.16207723	(p8-p7)/p7 : 0.1740696
n: 9	p9: 1.8527964	f(p9) : 0.70274056	(p9-p8)/p8 : 0.1275000
n: 10	p10: 2.2047137	f(p10) : 0.08361842	(p10-p9)/p9 : 0.1899385
n: 11	p11: 2.2522436	f(p11) : 0.06030405	(p11-p10)/p10 : 0.0215583
n: 12	p12: 2.3751824	f(p12) : 0.08732308	(p12-p11)/p11 : 0.0545850
n: 13	p13: 1.9778551	f(p13) : 0.40773432	(p13-p12)/p12 : 0.1672829
n: 14	p14: 2.4834677	f(p14) : 0.22767590	(p14-p13)/p13 : 0.2556368
n: 15	p15: 3.1227925	f(p15) : 3.81667523	(p15-p14)/p14 : 0.2574323
n: 16	p16: 2.4429108	f(p16) : 0.16144913	(p16-p15)/p15 : 0.2177159
n: 17	p17: 2.4128808	f(p17) : 0.12308091	(p17-p16)/p16 : 0.0122927
n: 18	p18: 2.3165481	f(p18) : 0.05796415	(p18-p17)/p17 : 0.0399244
n: 19	p19: 2.2307969	f(p19) : 0.06864113	(p19-p18)/p18 : 0.0370168
n: 20	p20: 2.7820825	f(p20) : 1.27695209	(p20-p19)/p19 : 0.2471250
n: 21	p21: 2.1994797	f(p21) : 0.08724733	(p21-p20)/p20 : 0.2094125
n: 22	p22: 2.1567544	f(p22) : 0.12437632	(p22-p21)/p21 : 0.0194252
n: 23	p23: 2.2998776	f(p23) : 0.05520794	(p23-p22)/p22 : 0.0663605
n: 24	p24: 2.4141139	f(p24) : 0.12448153	(p24-p23)/p23 : 0.0496706
n: 25	p25: 2.2088364	f(p25) : 0.08090520	(p25-p24)/p24 : 0.0850322
n: 26	p26: 1.8277117	f(p26) : 0.76911151	(p26-p25)/p25 : 0.1725455
n: 27	p27: 2.2536412	f(p27) : 0.05988824	(p27-p26)/p26 : 0.2330398
n: 28	p28: 2.2896076	f(p28) : 0.05470032	(p28-p27)/p27 : 0.0159592
n: 29	p29: 2.6688287	f(p29) : 0.75671127	(p29-p28)/p28 : 0.1656271
n: 30	p30: 2.2600589	f(p30) : 0.05818198	(p30-p29)/p29 : 0.1531645
n: 31	p31: 2.2260116	f(p31) : 0.07099620	(p31-p30)/p30 : 0.0150648
n: 32	p32: 2.4146480	f(p32) : 0.12509280	(p32-p31)/p31 : 0.0847419
n: 33	p33: 1.9784458	f(p33) : 0.40650444	(p33-p32)/p32 : 0.1806484
n: 34	p34: 2.6085481	f(p34) : 0.54222817	(p34-p33)/p33 : 0.3184835
n: 35	p35: 0.0912341	f(p35) : 3.99924165	(p35-p34)/p34 : 0.9650249
n: 36	p36: 3.0033856	f(p36) : 2.75729271	(p36-p35)/p35 : 31.9195406
n: 37	p37: 9.4687513	f(p37) : 0.05874056	(p37-p36)/p36 : 2.1526925
n: 38	p38: 9.6094857	f(p38) : 12.95950652	(p38-p37)/p37 : 0.0148630
n: 39	p39: 9.4693863	f(p39) : 0.00132615	(p39-p38)/p38 : 0.0145793
n: 40	p40: 9.4694007	f(p40) : 0.00002991	(p40-p39)/p39 : 0.0000015
n: 41	p41: 9.4694010	f(p41) : 0.00000000	(p41-p40)/p40 : 0.0000000

Tolerance: 10^{-6} , Approximation: 9.4694009967, Iterations: 41



Problem 2

Use Newton's method to approximate the zero of the function $f(x) = x^2 - 2e^{-x}x + e^{-2x}$ starting from $p_0 = 1$ using accurate to within 10^{-8} . What do you notice about the convergence of Newton's method? ==> Note that here the root has multiplicity > 1 , that is $f(r) = f'(r) = 0$ and hence Newton's method may not converge quadratically.

% Function

```
f = @(x) x.^2 - 2*x.*exp(-x) + exp(-2*x);
```

% Derivative

```
fprime = @(x) 2*(x - exp(-x)).*(1 + exp(-x));
```

% Plot f and f'

```
x = 0:0.01:2;
figure(2)
plot(x, f(x), 'b-', 'LineWidth', 2); hold on
plot(x, fprime(x), 'r-', 'LineWidth', 2)
legend('f(x)', 'f''(x)')
xlabel('x'); ylabel('y')
title('Function and its derivative')
grid on
```

% Parameters

```
tolexp = -10;           % tighter tolerance for better "true root"
maxiterations = 50;
p0 = 1;
```

```

[p_vals, n] = Newton_Scalar(f, fprime, tolexp, maxiterations, p0);

r = p_vals(n);

% True errors
errors_true = abs(p_vals(1:n) - r);
% Instead of measuring |pn-r| we could also measure |pn-p_(n+1)|, since
% near the root |pn-p_(n+1)| = |pn-r|

kmax = length(errors_true) - 10; % ignore last 3 points

e_n = errors_true(1:kmax-1);
e_np1 = errors_true(2:kmax);

figure
loglog(e_n, e_np1, 'o-', 'LineWidth', 2); hold on
xlabel('e_n')
ylabel('e_{n+1}')
title('Order of convergence (restricted range)')
grid on

% Rate of convergence

% pick a small middle segment
i1 = floor(length(e_n)/2);
i2 = i1 + 5;

x_ref = e_n(i1:i2);

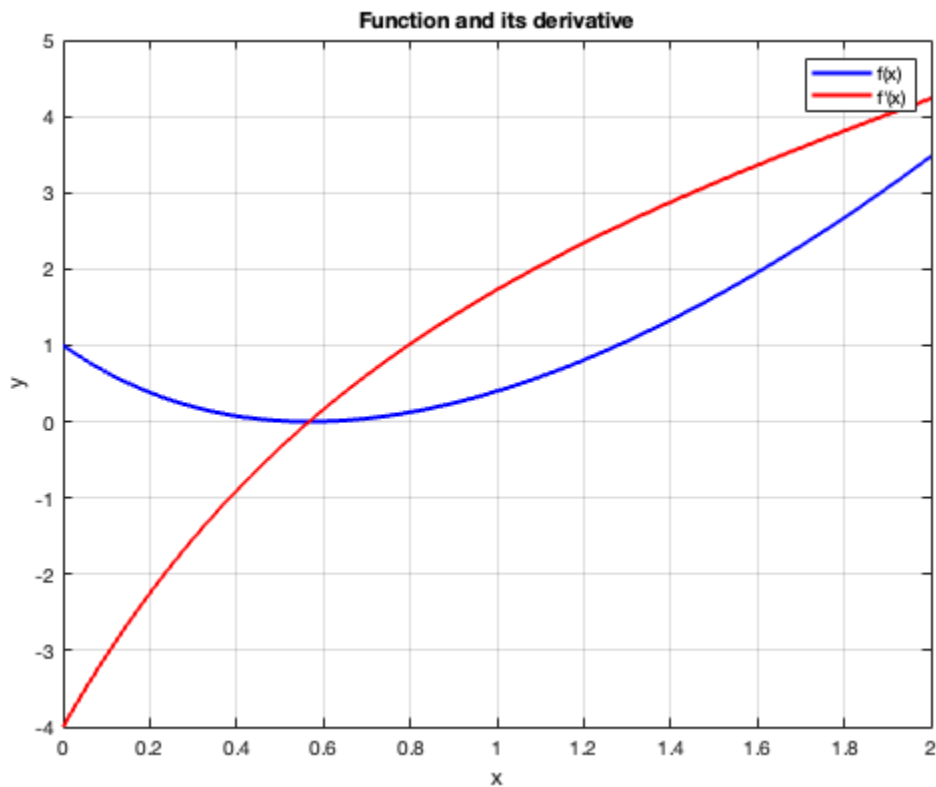
% slope 1: y = C*x (match locally)
C = e_np1(i1)/e_n(i1);
y_ref = 1e+1*C * x_ref;
figure(3)
loglog(x_ref, y_ref, 'k--', 'LineWidth', 2)
legend('Data', 'Slope = 1', 'Location', 'best')

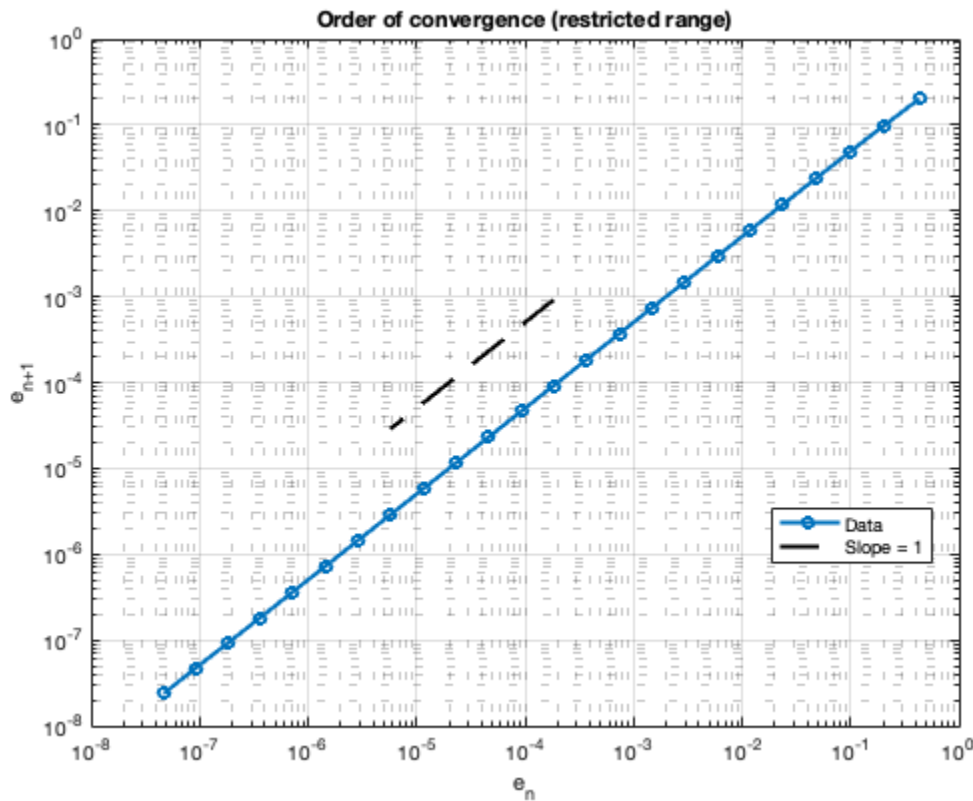
n: 0    p0: 1.000000000000
n: 1    p1: 0.76894142137    |(p1-p0)/p1|: 0.30048918189
n: 2    p2: 0.66458978661    |(p2-p1)/p2|: 0.15701660914
n: 3    p3: 0.61503323767    |(p3-p2)/p3|: 0.08057539967
n: 4    p4: 0.59088381394    |(p4-p3)/p4|: 0.04087000382
n: 5    p5: 0.57896292998    |(p5-p4)/p5|: 0.02059006431
n: 6    p6: 0.57304051622    |(p6-p5)/p6|: 0.01033506984
n: 7    p7: 0.57008876253    |(p7-p6)/p7|: 0.00517770895
n: 8    p8: 0.56861524224    |(p8-p7)/p8|: 0.00259141891
n: 9    p9: 0.56787907039    |(p9-p8)/p9|: 0.00129635320
n: 10   p10: 0.56751113143    |(p10-p9)/p10|: 0.00064833787
n: 11   p11: 0.56732719868    |(p11-p10)/p11|: 0.00032420929
n: 12   p12: 0.56723524148    |(p12-p11)/p12|: 0.00016211474
n: 13   p13: 0.56718926518    |(p13-p12)/p13|: 0.00008105990
n: 14   p14: 0.56716627761    |(p14-p13)/p14|: 0.00004053058
n: 15   p15: 0.56715478396    |(p15-p14)/p15|: 0.00002026545
n: 16   p16: 0.56714903717    |(p16-p15)/p16|: 0.00001013276

```

n: 17	p17: 0.56714616379	(p17-p16)/p17 : 0.00000506639
n: 18	p18: 0.56714472710	(p18-p17)/p18 : 0.00000253319
n: 19	p19: 0.56714400876	(p19-p18)/p19 : 0.00000126661
n: 20	p20: 0.56714364958	(p20-p19)/p20 : 0.00000063330
n: 21	p21: 0.56714346998	(p21-p20)/p21 : 0.00000031668
n: 22	p22: 0.56714338011	(p22-p21)/p22 : 0.00000015847
n: 23	p23: 0.56714333525	(p23-p22)/p23 : 0.00000007909
n: 24	p24: 0.56714331232	(p24-p23)/p24 : 0.00000004044
n: 25	p25: 0.56714330149	(p25-p24)/p25 : 0.00000001910
n: 26	p26: 0.56714329537	(p26-p25)/p26 : 0.00000001079
n: 27	p27: 0.56714329309	(p27-p26)/p27 : 0.00000000402
n: 28	p28: 0.56714329731	(p28-p27)/p28 : 0.00000000744
n: 29	p29: 0.56714329403	(p29-p28)/p29 : 0.00000000578
n: 30	p30: 0.56714329091	(p30-p29)/p30 : 0.00000000550
n: 31	p31: 0.56714331341	(p31-p30)/p31 : 0.00000003967
n: 32	p32: 0.56714330211	(p32-p31)/p32 : 0.00000001993
n: 33	p33: 0.56714329535	(p33-p32)/p33 : 0.00000001192
n: 34	p34: 0.56714328848	(p34-p33)/p34 : 0.00000001210
n: 35	p35: 0.56714328848	(p35-p34)/p35 : 0.00000000000

Tolerance: 10^{-10} , Approximation: 0.56714328848, Iterations: 35





Problem 3

Use Newton's method to find a solution to the following nonlinear system. Iterate until $|x^{(k)} - x^{(k-1)}| < 10^{-6}$. $6x_1 - 2\cos(x_2x_3) - 1 = 0$, $9x_2 + \sqrt{x_1^2 + \sin(x_3)} + 1.06 = 0$, $60x_3 + 3e^{-x_1x_2} + 10\pi - 3 = 0$.

```
x0 = [0;0;0]; tol = 1e-6; maxit = 50;
x = Newton_system(@f_fun, @Jacobian, x0, tol, maxit);
fprintf('\nFinal solution:\n');
disp(x)
```

```
f_fun(x)
%-----Functions-----%
function Secant(p0, p1, f, maxiterations, tolexp)
%function uses the Secant method to approximate zero of a function

%INPUT
% p0: first initial guess
% p1: second initial guess
% f: function for which we want to approximate a zero
% maxiterations: max number of iterations
% tolexp: exponent of stopping criteria tolerance

tolerance = 10^tolexp;
```

```

n = 2; %iteration counter
fp0 = f(p0); %computes function value of initial guesses
fp1 = f(p1);

fprintf(['n: 1\tp%i: %.' num2str(abs(tolexp)+1) 'f\t|f(p%i)|: %.8f\t|(p%i-
p%i)/p%i|: %.' num2str(abs(tolexp)+1) 'f\n'], n-1, p1, n-1, abs(fp1),n-1,
n-2, n-2, abs((p1-p0)/p0))

while n <= maxiterations
    %produces new approximation of root
    p = p1-fp1*(p1-p0)/(fp1-fp0);

    fprintf(['n: %i\tp%i: %.' num2str(abs(tolexp)+1) 'f\t|f(p%i)|: %.8f\t|
(p%i-p%i)/p%i|: %.' num2str(abs(tolexp)+1) 'f\n'], ...
        n, n, p, n, abs(f(p)), n, n-1, n-1, abs((p-p1)/p1))

    if abs((p-p1)/p1) < tolerance
        break
    end

    %updates the iteration counter
    n = n+1;

    %updates first guess with the second guess
    p0 = p1;
    fp0 = fp1;
    %updates second guess with the new approximation
    p1 = p;
    fp1 = f(p);
end

if n < maxiterations+1
    fprintf('Tolerance: 10^%i, Approximation: %.10f, Iterations: %i\n',
tolexp, p, n);
else
    fprintf('The max number of iterations %i was reached before
convergence\n', maxiterations);
end

%insert carriage return to separate output
fprintf('\n')
end

function [p_vals, n] = Newton_Scalar(f, fprime, tolexp, maxiterations, p0)

p_vals = zeros(maxiterations+1,1);
p_vals(1) = p0;

fprintf(['n: 0\tp0: %.' num2str(abs(tolexp)+1) 'f\n'], p0)

for n = 1:maxiterations

    p = p0 - f(p0)/fprime(p0);

```

```

        p_vals(n+1) = p;

        relerr = abs((p - p0)/p);

        fprintf(['n: %d\tp%d: %.' num2str(abs(tolexp)+1) ...
                'f\t|(p%d-p%d)/p%d|: %.' num2str(abs(tolexp)+1) 'f\n'], ...
                n, n, p, n, n-1, n, relerr)

        if relerr < 10^tolexp
            break
        end

        p0 = p;
    end

    fprintf('\nTolerance: 10^%d, Approximation: %.*f, Iterations: %d\n\n', ...
            tolexp, abs(tolexp)+1, p, n);

end
%
function x = Newton_system(Ffun, Jfun, x0, tol, maxit)
    x = x0;
    err = 1.0;
    it = 0;
    fprintf('k\t x1\t\t x2\t\t x3\t\t ||dx||_inf\n');
    while err > tol && it < maxit
        Fx = Ffun(x);
        Jx = Jfun(x);
        dx = Jx \ Fx;
        x = x - dx;
        err = max(abs(dx));
        it = it + 1;

        fprintf('%d\t% .6f\t% .6f\t% .6f\t% .2e\n', ...
                it, x(1), x(2), x(3), err);
    end
end
%
function F = f_fun(x)
    F = zeros(3,1);
    F(1) = 6*x(1) - 2*cos(x(2)*x(3)) - 1;
    F(2) = 9*x(2) + sqrt(x(1)^2 + sin(x(3)) + 1.06) + 0.9;
    F(3) = 60*x(3) + 3*exp(-x(1)*x(2)) + 10*pi - 3;
end
%
function J = Jacobian(x)
    J = zeros(3,3);
    % Row 1
    J(1,1) = 6;
    J(1,2) = 2*x(3)*sin(x(2)*x(3));
    J(1,3) = 2*x(2)*sin(x(2)*x(3));
    % Row 2
    denom = sqrt(x(1)^2 + sin(x(3)) + 1.06);
    J(2,1) = x(1)/denom;

```

```

J(2,2) = 9;
J(2,3) = cos(x(3))/(2*denom);
% Row 3
J(3,1) = -3*x(2)*exp(-x(1)*x(2));
J(3,2) = -3*x(1)*exp(-x(1)*x(2));
J(3,3) = 60;
end
%-----%

k      x1      x2      x3      ||dx||_inf
1      0.500000  -0.186142  -0.523599  5.24e-01
2      0.498158  -0.199607  -0.528826  1.35e-02
3      0.498145  -0.199606  -0.528826  1.31e-05
4      0.498145  -0.199606  -0.528826  7.49e-12

Final solution:
    0.4981
   -0.1996
   -0.5288

ans =

    1.0e-14 *

   -0.0222
    0.0111
   -0.3553

```

Published with MATLAB® R2024b